ML Pipeline

In [ ]:

```
Problem statement
Data collection
Data Reading(EDA)
Data cleaning
EDA
Data Transformation
Data spliting(80:20)
Model selection
Model Training
Model Evalution
Deployment
```

Linear Regression

Problem statement

In [121…

```
#House Price prediction

import numpy as np
import pandas as pd
```

In [123…

```
df=pd.read_csv(r"D:\Downloads\House Price Prediction Dataset.csv")
```

In [125…

```
df
```

Out[125…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	Price
0	1	1360	5	4	3	1970	Downtown	Excellent	No	149919
1	2	4272	5	4	3	1958	Downtown	Excellent	No	424998
2	3	3592	2	2	3	1938	Downtown	Good	No	266746
3	4	966	4	2	2	1902	Suburban	Fair	Yes	244020
4	5	4926	1	4	2	1975	Downtown	Fair	Yes	636056
...	...	...	...	...	...	...	...	...	...	...
1995	1996	4994	5	4	3	1923	Suburban	Poor	No	295620
1996	1997	3046	5	2	1	2019	Suburban	Poor	Yes	580929
1997	1998	1062	5	1	2	1903	Rural	Poor	No	476925
1998	1999	4062	3	1	2	1936	Urban	Excellent	Yes	161119
1999	2000	2989	5	1	3	1903	Suburban	Fair	No	482525

2000 rows × 10 columns

In [127…

```
#check dataset information

df.shape
```

Out[127…

```
(2000, 10)
```

In [129…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   Id         2000 non-null   int64 
 1   Area       2000 non-null   int64 
 2   Bedrooms   2000 non-null   int64 
 3   Bathrooms  2000 non-null   int64 
 4   Floors     2000 non-null   int64 
 5   YearBuilt  2000 non-null   int64 
 6   Location   2000 non-null   object
 7   Condition  2000 non-null   object
 8   Garage     2000 non-null   object
 9   Price      2000 non-null   int64 
dtypes: int64(7), object(3)
memory usage: 156.4+ KB
```

In [131…

```
df.isna().sum()
```

Out[131…

```
Id           0
Area         0
Bedrooms     0
Bathrooms    0
Floors       0
YearBuilt    0
Location     0
Condition    0
Garage       0
Price        0
dtype: int64
```

Define Features and Target

In [134…

```
Y=df[['Price']]
X=df.drop(labels=['Price'],axis=1)
```

In [136…

```
Y
```

Out[136…

	Price
0	149919
1	424998
2	266746
3	244020
4	636056
...	...
1995	295620
1996	580929
1997	476925
1998	161119
1999	482525

2000 rows × 1 columns

In [138…

```
X
```

Out[138…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage
0	1	1360	5	4	3	1970	Downtown	Excellent	No
1	2	4272	5	4	3	1958	Downtown	Excellent	No
2	3	3592	2	2	3	1938	Downtown	Good	No
3	4	966	4	2	2	1902	Suburban	Fair	Yes
4	5	4926	1	4	2	1975	Downtown	Fair	Yes
...	...	...	...	...	...	...	...	...	...
1995	1996	4994	5	4	3	1923	Suburban	Poor	No
1996	1997	3046	5	2	1	2019	Suburban	Poor	Yes
1997	1998	1062	5	1	2	1903	Rural	Poor	No
1998	1999	4062	3	1	2	1936	Urban	Excellent	Yes
1999	2000	2989	5	1	3	1903	Suburban	Fair	No

2000 rows × 9 columns

Separate Numerical and Categorical columns

In [141…

```
cat=[]
con=[]
for i in X.columns:
    if df[i].dtypes=="object":
        cat.append(i)
    else:
        con.append(i)
```

In [143…

```
cat
```

Out[143…

```
['Location', 'Condition', 'Garage']
```

In [145…

```
con
```

Out[145…

```
['Id', 'Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']
```

In [147…

```
categorical_columns=df[cat]
```

In [149…

```
categorical_columns
```

Out[149…

	Location	Condition	Garage
0	Downtown	Excellent	No
1	Downtown	Excellent	No
2	Downtown	Good	No
3	Suburban	Fair	Yes
4	Downtown	Fair	Yes
...	...	...	...
1995	Suburban	Poor	No
1996	Suburban	Poor	Yes
1997	Rural	Poor	No
1998	Urban	Excellent	Yes
1999	Suburban	Fair	No

2000 rows × 3 columns

In [151…

```
numerical_columns=df[con]
```

In [153…

```
numerical_columns
```

Out[153…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt
0	1	1360	5	4	3	1970
1	2	4272	5	4	3	1958
2	3	3592	2	2	3	1938
3	4	966	4	2	2	1902
4	5	4926	1	4	2	1975
...	...	...	...	...	...	...
1995	1996	4994	5	4	3	1923
1996	1997	3046	5	2	1	2019
1997	1998	1062	5	1	2	1903
1998	1999	4062	3	1	2	1936
1999	2000	2989	5	1	3	1903

2000 rows × 6 columns

In [ ]:

In [156…

```
from sklearn.preprocessing import OneHotEncoder

ohe=OneHotEncoder(sparse_output=False)
encoder=ohe.fit_transform(categorical_columns)
```

In [158…

```
encoder
```

Out[158…

```
array([[1., 0., 0., ..., 0., 1., 0.],
       [1., 0., 0., ..., 0., 1., 0.],
       [1., 0., 0., ..., 0., 1., 0.],
       ...,
       [0., 1., 0., ..., 1., 1., 0.],
       [0., 0., 0., ..., 0., 0., 1.],
       [0., 0., 1., ..., 0., 1., 0.]])
```

In [160…

```
ohe.get_feature_names_out()
```

Out[160…

```
array(['Location_Downtown', 'Location_Rural', 'Location_Suburban',
       'Location_Urban', 'Condition_Excellent', 'Condition_Fair',
       'Condition_Good', 'Condition_Poor', 'Garage_No', 'Garage_Yes'],
      dtype=object)
```

In [162…

```
ohe_df=pd.DataFrame(encoder,columns=ohe.get_feature_names_out().tolist())
```

In [164…

```
new_df=pd.concat([numerical_columns,ohe_df],axis=1)
```

In [166…

```
new_df
```

Out[166…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location_Downtown	Location_Rural	Location_Suburban	Location_Urban	Condition_Excellent	Condition_Fair	Condition_Good	Condition_Poor	Garag
0	1	1360	5	4	3	1970	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	
1	2	4272	5	4	3	1958	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	
2	3	3592	2	2	3	1938	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
3	4	966	4	2	2	1902	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	
4	5	4926	1	4	2	1975	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1995	1996	4994	5	4	3	1923	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	
1996	1997	3046	5	2	1	2019	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	
1997	1998	1062	5	1	2	1903	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	
1998	1999	4062	3	1	2	1936	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	
1999	2000	2989	5	1	3	1903	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	

2000 rows × 16 columns

In [168…

```
ohe.get_feature_names_out().tolist()
```

Out[168…

```
['Location_Downtown',
 'Location_Rural',
 'Location_Suburban',
 'Location_Urban',
 'Condition_Excellent',
 'Condition_Fair',
 'Condition_Good',
 'Condition_Poor',
 'Garage_No',
 'Garage_Yes']
```

In [170…

```
X
```

Out[170…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage
0	1	1360	5	4	3	1970	Downtown	Excellent	No
1	2	4272	5	4	3	1958	Downtown	Excellent	No
2	3	3592	2	2	3	1938	Downtown	Good	No
3	4	966	4	2	2	1902	Suburban	Fair	Yes
4	5	4926	1	4	2	1975	Downtown	Fair	Yes
...	...	...	...	...	...	...	...	...	...
1995	1996	4994	5	4	3	1923	Suburban	Poor	No
1996	1997	3046	5	2	1	2019	Suburban	Poor	Yes
1997	1998	1062	5	1	2	1903	Rural	Poor	No
1998	1999	4062	3	1	2	1936	Urban	Excellent	Yes
1999	2000	2989	5	1	3	1903	Suburban	Fair	No

2000 rows × 9 columns

In [172…

```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(new_df,Y,test_size=0.2,random_state=42)
```

In [174…

```
xtrain
```

Out[174…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location_Downtown	Location_Rural	Location_Suburban	Location_Urban	Condition_Excellent	Condition_Fair	Condition_Good	Condition_Poor	Garag
968	969	4483	4	4	3	1933	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	
240	241	1062	3	3	1	1970	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
819	820	1422	3	4	1	1993	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	
692	693	2658	2	3	1	1972	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	
420	421	3286	2	4	1	1981	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1130	1131	3483	5	3	3	1931	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
1294	1295	1747	3	3	2	1941	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	
860	861	1484	2	3	3	1973	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
1459	1460	3456	5	4	1	1955	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
1126	1127	1682	2	1	3	2006	0.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	

1600 rows × 16 columns

In [176…

```
xtest
```

Out[176…

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location_Downtown	Location_Rural	Location_Suburban	Location_Urban	Condition_Excellent	Condition_Fair	Condition_Good	Condition_Poor	Garag
1860	1861	633	1	4	2	1901	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	
353	354	3856	1	1	2	1901	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	
1333	1334	1598	5	4	1	2013	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	
905	906	1705	3	1	2	1996	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
1289	1290	2564	5	2	3	1994	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
965	966	1811	1	3	3	1970	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	
1284	1285	794	3	2	2	2013	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	
1739	1740	3540	4	4	2	1939	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	
261	262	3670	5	2	2	1910	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
535	536	3519	5	1	3	1938	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	

400 rows × 16 columns

In [178…

```
ytrain
```

Out[178…

	Price
968	434325
240	614772
819	922811
692	794314
420	796988
...	...
1130	387088
1294	942215
860	323139
1459	274997
1126	644206

1600 rows × 1 columns

In [180…

```
ytest
```

Out[180…

	Price
1860	514764